

Рассмотрим темы:

- ✓ Методы исследования продукта
- ✓ Оформление тестов (тест-кейсы, чек листы)
- ✓ Классификация ошибок
- ✓ Жизненный цикл бага
- ✓ Отчеты об ошибках
- ✓ Отчеты о результатах тестирования

Исследование продукта

Задача: *протестировать уже готовый продукт.*

Что делать?

- ✓ исследовать продукт - что он умеет;
- ✓ правильно задавать вопросы;
- ✓ использовать инструменты для записи результатов.

С чего начинаем исследование?

- ✓ Спецификация есть – изучаем ее
- ✓ Спецификации нет – задаем вопросы

Как правильно задавать вопросы

Вопросы бывают открытые и закрытые .

Закрытые вопросы	Открытые вопросы
<i>Возможен только однозначный ответ:</i> дата; время; название; количество; ДА или НЕТ? Например: Сколько лет вы работаете в тестировании? Вы проводили нагрузочное тестирование? Система должна обрабатывать TXT-файлы?	<i>Нужен развернутый ответ</i> Открытые вопросы обычно начинаются со слов: Что? Как? Почему? Каким образом? При каких условиях?

Фактически закрытый вопрос означает, что вы сами уже додумали себе требования и их же и будете тестировать. А тестировщик НИКОГДА не додумывает. Ведь иначе он будет тестировать вообще не то, что нужно.

Например, система принимает на входе файл с текстом и проверяет его на опечатки.

Тестировщик додумал себе, что это простой блокнот, и уточняет у заказчика:

- Система должна обрабатывать TXT -файлы?

-Да, должна.

- Ок, спасибо.

Полностью уверенный в том, что узнал всё, что ему было нужно, тестировщик готовит свои тесты.

А на самом деле заказчик ждет, что система будет уметь работать со всеми текстовыми форматами, включая Excel, а еще она сможет считывать текст с картинок! И именно в этом ее фишка. То есть главный функционал. А тестировщик картинки с текстом вообще не проверяет, так как считает это негативным тестированием.

- ✓ Всегда старайтесь задавать максимально открытый вопрос.
- ✓ Если видите, что ваш вопрос «закрытый», подумайте, что именно вы хотели узнать об этом и спросите.

Примеры:

Хотим узнать, какие форматы принимает система.

Закрытый вопрос: Система должна обрабатывать TXT-файлы?

А на самом деле, кроме TXT, система еще должна уметь распознавать текст с картинок. Но вы же спрашивали только про TXT, про него ответ и получили.

Поэтому спрашивать лучше так:

Открытый вопрос: Какой формат должна обрабатывать система?

Хотим узнать, есть ли у машины автозапуск.

Закрытый вопрос: Можно ли завести машину с брелка?

На самом же деле можно запустить машину, позвонив ей по телефону или отправив СМС. Но вы додумали требование про брелок и только про него и спросили.

Поэтому спрашивать лучше так:

Открытый вопрос: Есть ли у машины автозапуск? Какой?

Как оформить результат, чтобы не забыть всё, что мы узнали?

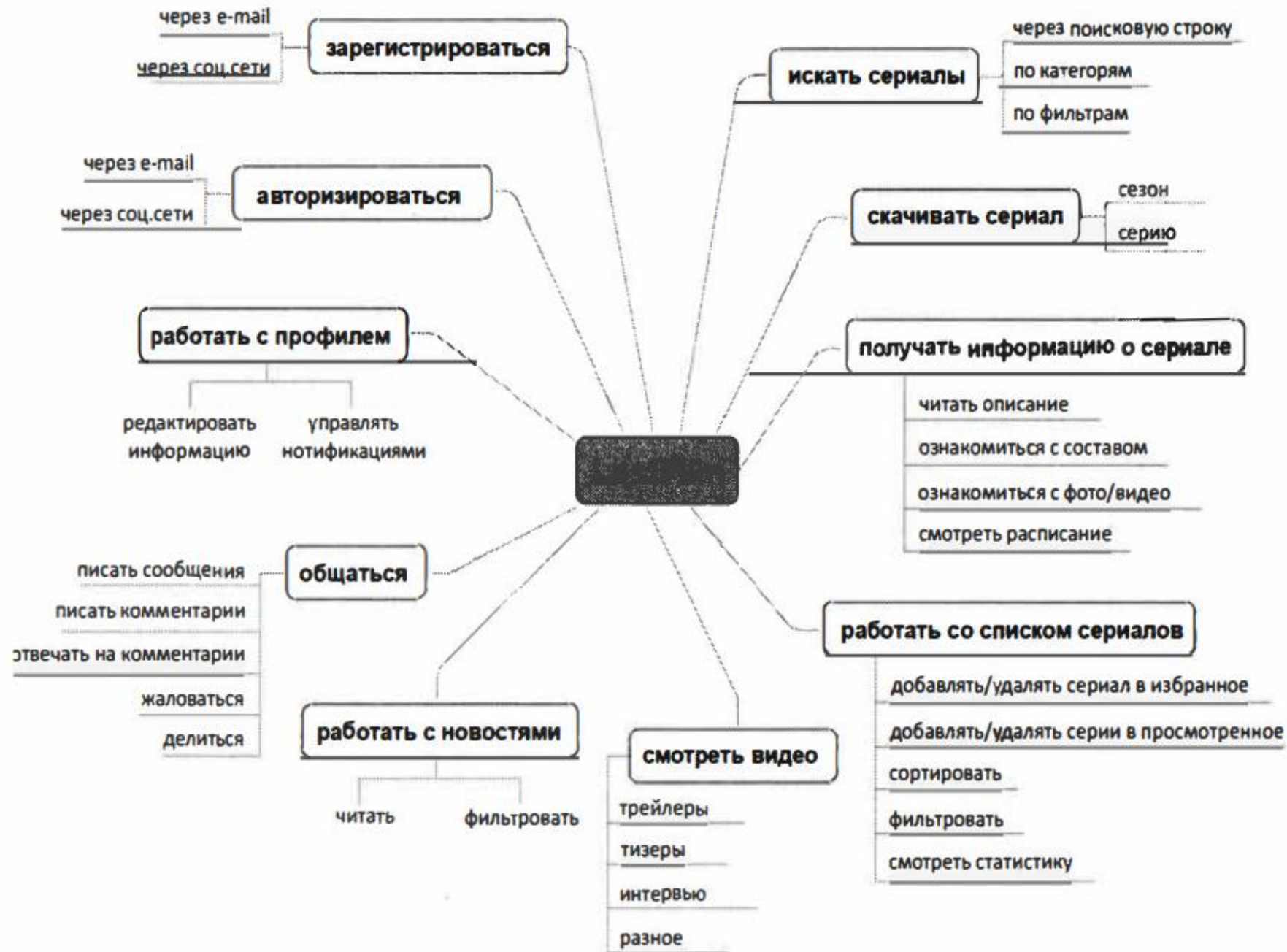
- ✓ Записать в блокнот
- ✓ Нарисовать карту приложения

Как нарисовать карту приложения?

Есть несколько основных правил, которые помогут сделать карту простой и понятной.

Выделите основные функции, задав себе вопросы:

- ❖ *зачем пользователю наш продукт?*
- ❖ *что он там делает?*
- ❖ *что мы хотим, чтобы он делал?*



По важности

Регистрация
здесь не самая
важная функция,
поэтому она в
конце

После изучения функционала продукта

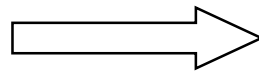
- проектируем тесты;
- оформляем тест-кейсы;
- пишем чек-листы.

Проектирование тестов

При продумывании тестов нужно помнить о том, каким должен быть тест и о приоритетах выполнения.

Каким должен быть тест?

- Тест должен быть конкретным
- У теста должен быть результат



Из описания теста должно быть понятно:

- что сделать;
- как сделать.

Приоритеты выполнения тестов

ВСЕГДА начинаем с позитивного тестирования и только потом проводим негативное.

Потому что задача тестировщика - не найти как можно больше багов, а дать наиболее полную информацию по приложению

Как оформлять тесты?

1. Тест-кейс (ТК) - это подробное описание проверки.

ТК1. Авторизация через email

Предварительные шаги

Зарегистрироваться с email test@test.ru и паролем 1

Шаги

1. Открыть сайт <https://www.example.com/>.
2. Нажать на кнопку «Вход» в правом верхнем углу окна сайта.
3. В поле «Email» ввести значение: test@test.ru.
4. В поле «Пароль» ввести значение: 1.
5. Нажать кнопку «Войти».

Ожидаемый результат

- Открылась главная страница сайта.
- В верхнем правом углу страницы отображается значок личного кабинета (недоступен без авторизации)

Структура тест-кейса

1. шаги — это инструкция по вводу;
2. исполнение шагов — это ввод;
3. ожидаемый результат;
4. фактический результат.

Исполнение тест-кейса завершается сравнением фактического результата и ожидаемого результата:

- ✓ Положительный исход (**PASS**), если ФР равен ОР,
- ✗ Отрицательный исход (**FAIL**), если ФР не равен ОР: найден баг!

Иногда возникает ситуация, когда мы заблокированы, так как не можем пройти ВСЕ шаги тест-кейса. Например, мы не можем продвинуться дальше, если кнопки "Завершить заказ" из шага 14 не существует на соответствующей веб-странице. В таком случае мы рапортуем баг (в данном случае баг об отсутствии кнопки "Завершить заказ") и откладываем исполнение тест-кейса до устранения бага.

Атрибуты тест-кейса (рекомендуемые)

- **Уникальный идентификатор тест-кейса.**
- **Приоритет** — важность по шкале от 1 (высший приоритет) до n (низший приоритет).
- **Название** — основная тема, или идея тест-кейса. Краткое описание его сути.
- **Предусловия** — описание условий, которые не имеют прямого отношения к проверяемому функционалу, но должны быть выполнены. Например, «пользователь авторизован».
- **Постусловия** — действия, которые нужно сделать после проведения проверки. Например, удалить внесённые данные из БД.
- **Окружение** — на каком устройстве, в каком браузере. Иногда заполняют до тестирования, иногда — сам тестирующий указывает после.
- **История редактирования** — кто создал/изменил? Когда? Зачем? Почему?
- **Шаги** — описание последовательности действий, которая должна привести нас к ожидаемому результату
- **Ожидаемый результат** — что мы ожидаем увидеть после выполнения шагов.

TC ID/Priority	CCPG0001	1
IDEA: Оплата может быть произведена картой VISA SETUP and ADDITIONAL INFO: Эккаунт: testuser1/paSSwOrd Наименование товара: book117 Данные карты: Номер: 9999-5148-2222-1277 Окончание действия: 12/07 CVV2: 778 SQL1: select result from cc_transaction where id = <номер заказа>;		
Revision History		
Created on: 11/17/2003 by O.Тарасов	Новый тест-кейс	

Пример оформления тест-кейса
из книги Романа Савина
«Тестирование Дот Ком»

Execution part	
PROCEDURE	EXPECTED RESULT
1. Открой www.main.testshop.rs 2. Введи имя пользователя. 3. Введи пароль. 4. Нажми кнопку "Войти". 5. Введи наименование товара в поле поиска. 6. Нажми кнопку "Найти". 7. Кликни линк "Добавить в корзину". 8. Кликни линк "Корзина". 9. Кликни линк "Оплатить". 10. Выбери вид карты. 11. Введи номер карты. 12. Введи срок окончания действия. 13. Введи CVV2. 14. Нажми кнопку "Завершить заказ". 15. Запиши номер заказа	> "10"
16. Запроси базу данных с SQL1 и запиши результат	

FAILED #33070 Множественное добавление одного товара в корзину

Overview History Retries Attachments

Message

Отсутствует задержка в 1 сек. между добавлением товаров (добавляются со скоростью клика, проверялось на автокликах раз в 0,1 сек.). Товаров на складе в БД было 1000 штук, в корзину добавилось больше этого значения, после чего элементы страницы перестали реагировать на любые действия. После обновления страницы корзина стала пустой.

Комментарий к статусу

Similar failures

No content

Description

No description

Precondition

Открыт каталог товаров

Execution



1 Напротив товара арт.123456 нажать на "+" много раз подряд (например, подключить автокликер)

5s 310ms

Expected result

Товары добавляются в корзину с задержкой 1 сек, максимальное количество товаров ограничено количеством на складе (данные из БД), общая стоимость корзины пересчитывается в режиме реального времени и отображается корректно в зависимости от количества товара

Comments



2023-03-14 18:33:12
11s 026ms

Tags

WEB КОРЗИНА



Issues links

ACCANL-7

Test keys

Members

Owner:
OErmolaeva



Environment

No environment

Fields

Section1:
Корзина

Ссылка на баг

Тест-кейсы м.б.:

- позитивные
- негативные
- деструктивные

Пример деструктивного
тест-кейса

Особенности тест-кейсов

- **В них максимум информации.** Их легко можно отдать новичку, и он проведет проверки, не задавая вопросов.
- **Они независимые.** Хорошие тест-кейсы не зависят друг от друга, поэтому можно легко редактировать один, не боясь, что развалятся все остальные.
- **Они небольшие.** Один тест-кейс – одна проверка.

Тест-кейсы нужны, когда у нас:

- **жизненно важные системы**, ошибка в которых может привести к гибели людей (авиастроение, медицина, ПО для атомных станций). Надо тестировать очень тщательно;
- **сложная система или сложная часть системы.** Чтобы каждый раз не вспоминать «а как мне это сделать?», лучше написать тест-кейс.
- **в команде постоянно появляются новые люди.**

Тест-кейсы не нужны, когда у нас:

- **простые системы** (веб-сайты одностраничники, мобильные приложения и т. п.);
- **если в команде всего один или два тестировщика, знающие свой продукт.** Время, потраченное на создание и поддержку тест-кейсов, никогда не окупится.

Так как тест-кейсы очень сложно поддерживать, то чаще используют чек-листы или комбинацию «чек-листы & тест-кейсы».

В последнем случае большинство проверок пишут в виде чек-листов, а особо сложные уже в виде тест-кейсов, чтобы каждый раз не вспоминать, как этот хитрый сценарий работает.

2 . Чек–листы

Чек-лист - это список проверок. Это своего рода напоминание: «не забудь проверить то и то».

Пример чек-листов в жизни:

- ✓ список покупок
- ✓ список вещей в отпуск

Как оформить чек-лист?

Оформление может
быть любым:

- список
- таблица
- схема
-

Имена		
Мужское		
Женское	✓	
Унисекс	✓	
Составное		
Пустое		

Проверка	Пример	Результат
Мужское	Иван	Успешная регистрация
Женское	Мария	Успешная регистрация
Унисекс	Саша	Успешная регистрация
Пустое		Ошибка: введите имя!

Это чек-лист

И это чек-лист

И даже это чек-лист



Оформление чек-листа

Неправильно	Слишком подробно	Правильно
1. Открыть страницу регистрации https://www.example.com/. Ввести в поле «имя» обычное имя, в поле «емайл» — корректный емайл, в поле «пароль» — корректный пароль. 2. Открыть страницу регистрации https://www.example.com/. Ввести в поле «имя» распространенное имя, в поле «емайл» — корректный емайл, в поле «пароль» — корректный пароль. 3. Открыть страницу регистрации https://www.example.com/. Ввести в поле «имя» редкое имя, в поле «емайл» — корректный емайл, в поле «пароль» — корректный пароль. 4. ...	Открыть страницу регистрации https://www.example.com/. Проверки для поля «имя»: 1. Обычное. 2. Распространенное. 3. Редкое. 4. ...	

Лучше сразу писать
конкретные
примеры

Если «Регистрация прошла успешно, на почту отправлено письмо-приветствие» - результат «ОК». (вынесли из таблицы общий результат, что бы не засорять таблицу)



Описание	Пример	Результат
Все поля заполнены правильно	Имя: Ольга Email: ok.molechka@gmail.com Пароль: 1	ОК
Проверка поля «Имя»		
Свое имя	Ольга	ОК
Короткое имя	Ева	ОК
Длинное имя (составное)	Розалинд Аруша Аркадина Алталун Флоренс Луна	ОК
...	...	...
Проверка поля «Email»		
Корректный email (популярный домен)	olga@mail.ru	ОК
Точка внутри email	ok.molechka@gmail.com	ОК
Кириллический email	олечка@мустики.рф	ОК
Пустая почта		Ошибка
Одно слово вместо домена	olga@fdgfdg	Ошибка
...		

Но лучше результат описывать конкретно, или добавить графу для комментариев.



Описание	Пример	Результат
Все поля заполнены правильно	Имя: Ольга Email: ok.molechka@gmail.com Пароль: 1	Регистрация прошла успешно, на почту отправлено письмо-приветствие

Проверка поля «Имя»		
Свое имя	Ольга	Успешная регистрация
Короткое имя	Ян	Ограничений на минимальное имя нет
Длинное имя (составное)	Розалинд Аруша Аркадина Алталун Флоренс Луна	В личном кабинете имя отображается нормально, влезает в отведенное под него место
...	...	...

Особенности чек-листов

- минимум информации.
- независимые.

Плюсы

- Меньше текста - проще и быстрее читать.
- Проще поддерживать - за счет предыдущих плюсов в основном. Если меньше текста, он медленнее устаревает.
- Быстрее писать - аналогично: меньше текста, больше дела!

Минусы

- Не все поймут, как проводить тест.

Чек-листы применимы, когда:

- вы - единственный тестирующий на проекте. Тогда нет смысла тратить время на подробное описание шагов, которые вы и так уже все наизусть знаете;
- система не суперсложная, и без указания конкретных шагов понятно, что и как делать.

Есть еще ЧИТ-ЛИСТЫ 😊

- Чек-лист - список проверок
- Чит-лист - список универсальных проверок

Подробнее о чит-листах

<https://habr.com/ru/articles/733986/>

<https://habr.com/ru/articles/715262/>

Чит-лист для проверки поля email

- 1) Пустое поле email -> Сообщение о незаполненном поле email
- 2) Email в нижнем регистре -> Операция проводится успешно
- 3) Email в верхнем регистре -> Операция проводится успешно
- 4) Email с цифрами в имени аккаунта -> Операция проводится успешно
- 5) Email с цифрами в доменной части -> Операция проводится успешно
- 6) Email с дефисом в имени аккаунта -> Операция проводится успешно
- 7) Email с дефисом в доменной части -> Операция проводится успешно
- 8) Email со знаком подчеркивания в имени аккаунта -> Операция проводится успешно
- 9) Email со знаком подчеркивания в доменной части -> Операция проводится успешно
- 10) Email с точками в имени аккаунта -> Операция проводится успешно
- 11) Email с несколькими точками в доменной части -> Операция проводится успешно
- 12) Email без точек в доменной части -> Должно появиться сообщение о неправильном или некорректном e-mail введенном в поле
- 13) Превышение длины email (>320 символов) -> Должно появиться сообщение о неправильном или некорректном e-mail введенном в поле
- 14) Отсутствие @ в email -> Должно появиться сообщение о неправильном или некорректном e-mail введенном в поле
- 15) Email с пробелами в имени аккаунта -> Должно появиться сообщение о неправильном или некорректном e-mail введенном в поле.

ИТАК.....

Тест-кейс

Шаги

1. Открываем сайт <https://www.example.com/>
2. Нажимаем кнопку «Вход» в правом верхнем углу.
3. Устанавливаем курсор на поле «email».
4. Вводим значение «olga@mail.com».
5. Устанавливаем курсор на поле «Пароль».
6. Вводим значение «12345».
7. Нажимаем «Сохранить».

Ожидаемый результат

Открылась главная страница сайта.

В верхнем правом углу отображается приветствие — «Здравствуйте, Ольга!»

Чек-лист

- Авторизоваться через email

Классификация ошибок
Жизненный цикл бага
Отчеты об ошибках

Что такое баг?



Баг (*bug*) — это отклонение фактического результата (*actual result*) от ожидаемого результата (*expected result*).

Баг живет лишь при **одновременном** выполнении всех трех условий:

1. **Известен** фактический результат;
2. **Известен** ожидаемый результат;
3. **Известно**, что результат из пункта 1 не равен результату из пункта 2.

Источники ожидаемого результата

- ✓ **Спецификация ! ! !**
- ✓ жизненный опыт
- ✓ здравый смысл
- ✓ устоявшиеся стандарты
- ✓ статистические данные
- ✓ авторитетное мнение и др.

Статистические данные:

Время ожидания открытия веб-страницы, после которого пользователь уходит

!!! В большинстве случаев баг — это отклонение от спецификации

Здравый смысл

Пример: по спецификации пользователь может загрузить лишь одну фотографию за раз. А что, если у него таких фотографий 200? Будет он счастлив? Что делаем? Пишем *Feature Request* ("запрос об улучшении").

УСТОЯВШИЕСЯ СТАНДАРТЫ

Как правило, после регистрации, пользователь должен получить е-мейл с подтверждением. Если спек не упоминает о таком е-мейле, вы можете потребовать дополнить его на основании сложившейся практики.

Пример выявления багов

Пункт 19.а спецификации #8724 "О регистрации нового пользователя" устанавливает:
«Поле "Имя" должно быть обязательным. Страница с ошибкой должна быть показана, если пользователь посылает регистрационную форму без заполнения указанного поля».

Функциональный баг

В общем все просто:

- тестировщик идет на страничку с регистрационной формой;
- кликает линк "Регистрация";
- заполняет все обязательные поля, **кроме поля "Имя"**;
- нажимает на кнопку "Зарегистрироваться".

Если ошибка не показана и регистрация подтверждается, то нужно рапортовать баг.

Баг в спецификации

! НО. Непонятно, каким должно быть сообщение об ошибке !

Программист может написать любое на своё усмотрение.

Ошибка, дефект (баг) и отказ в контексте жизненного цикла дефекта

- ✓ **Ошибка (error)** — например когда разработчик видит отличие между тем как приложение себя ведет и тем как должно себя вести, в процессе разработки
- ✓ **Дефект (баг)** — случается когда тестировщик обнаруживает несоответствие между реальным и предполагаемым поведением приложения в процессе тестирования
- ✓ **Отказ (failure)** — несоответствие реального и предполагаемого поведения обнаруживается уже в процессе пользования приложением конечным пользователем, клиентом, или тестировщиком на этапе приемочного тестирования.

Типы багов

В терминологии тестирования программного обеспечения существует множество типов багов, которые можно классифицировать по разным критериям. Вот некоторые из типов багов:

- ✓ **Воспроизводимый** — баг, который может быть воспроизведен тестировщиком в тестовом окружении.
- ✓ **Не воспроизводимый** — баг, который не может быть воспроизведен тестировщиком в тестовом окружении.
- ✓ **Фатальный** — баг, который вызывает крах или аварийное завершение программы.
- ✓ **Не фатальный** — баг, который не вызывает крах программы, но может приводить к некорректному поведению или неправильным результатам.
- ✓ **Функциональный** — баг, связанный с неправильной работой функциональности программы.
- ✓ **Нефункциональный** — баг, связанный с аспектами производительности, надежности, безопасности и т.д. программы.
- ✓ **Интерфейсный** — баг, связанный с интерфейсом пользователя.
- ✓ **Синтаксический** — баг, связанный с нарушением правил синтаксиса языка программирования.
- ✓ **Логический** — баг, связанный с неправильной логикой работы программы.
- ✓ **Спецификационный** — баг, связанный с нарушением требований к программе, указанных в спецификации.
- ✓ **Неоднозначный** — баг, который может иметь несколько причин и неоднозначный результат.

Вид бага	Что означает	Пример
Функциональный	Баг, при котором действия пользователя не совпадают с их результатом	Пользователь нажимает на кнопку «Личный кабинет», а попадает в раздел «Корзина»
Визуальный	Баг, из-за которого деформируется интерфейс приложения или веб-страницы	Кнопка и её надпись разъехались в разные стороны
Логический	Баг, при котором приложение или сайт нарушают логику	Сайт разрешает установить слишком старую дату рождения, например 1700 год. Или, наоборот, дату из будущего
Баг нагрузки	Баг, при котором приложение или сайт не справляются с большим количеством входящего трафика	Приложение социальной сети не работает или зависает из-за наплыва пользователей
Баг производительности	Баг, который делает приложение «прожорливым» до памяти, заряда и т. д.	Во время сессий в мобильной игре смартфон разряжается гораздо быстрее, чем в других играх

СЕРЬЕЗНОСТЬ БАГА

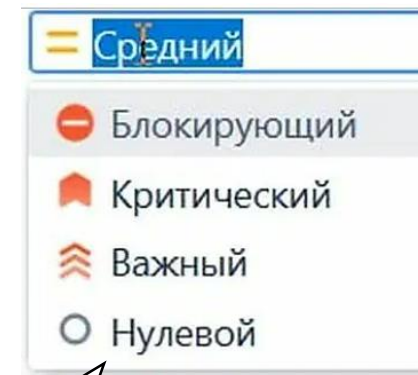
Серьезность (Severity) – это то, насколько баг влияет на систему

- ✓ Блокирующий (Blocker);
- ✓ Критичный (Critical);
- ✓ Значительный (Major);
- ✓ Незначительный (Minor);
- ✓ Тривиальный (Trivial).

ПРИОРИТЕТ БАГА

Приоритет (Priority) – это то, насколько срочно надо исправить баг

- ✓ Высокий (High);
- ✓ Средний (Medium);
- ✓ Низкий (Low).



Могут быть и другие приоритеты

У бага могут не совпадать уровень приоритетности и серьезности.

Например, замечена опечатка на главной странице Google.com. **Приоритет исправления такого бага будет высокий**, потому что его заметит много людей и пострадает репутация компании. А **серьезность бага – незначительная**, потому что это всего лишь опечатка, не влияющая на работу сайта.

Примеры по уровням важности

Серьёзность бага	Как влияет на систему
S1 — Блокирующая (Blocker).	Приложение не запускается ни на одном устройстве или выдаёт ошибку при загрузке. <i>Пример: приложение не открывается, поэтому невозможно сделать покупку.</i>
S2 — Критическая (Critical).	Часть функционала не работает, но это не блокирует процесс. <i>Пример: Скидка по купону не работает, но товар всё равно можно купить без скидки.</i>
S3 — Значительная (Major).	Часть логики работает некорректно, но пользователь может решить проблему другим способом. <i>Пример: поиск в приложении не выдаёт нужный товар, но его можно найти, если искать по брендам.</i>
S4 — Незначительная (Minor).	Не нарушает логику приложения. <i>Пример: кнопка «Купить» уехала за экран и видна только наполовину.</i>
S5 — Тривиальная (Trivial).	Не относится к логике приложения и не влияет на общее качество продукта. <i>Пример: малозаметная опечатка в меню.</i>

Жизненный цикл дефекта(бага) – это цикл, через который проходит дефект за весь период своего существования, включая его различные состояния.

Он начинается с момента обнаружения тестировщиком нового бага, и заканчивается, когда тестировщик закрывает баг, гарантируя, что он не будет воспроизведен снова.

Некоторые этапы цикла могут зависеть от компании и принятых в ней процедур, методик тестирования и инструментов, а главным образом от используемого баг-трекера (поэтому схемы в разных источниках отличаются).

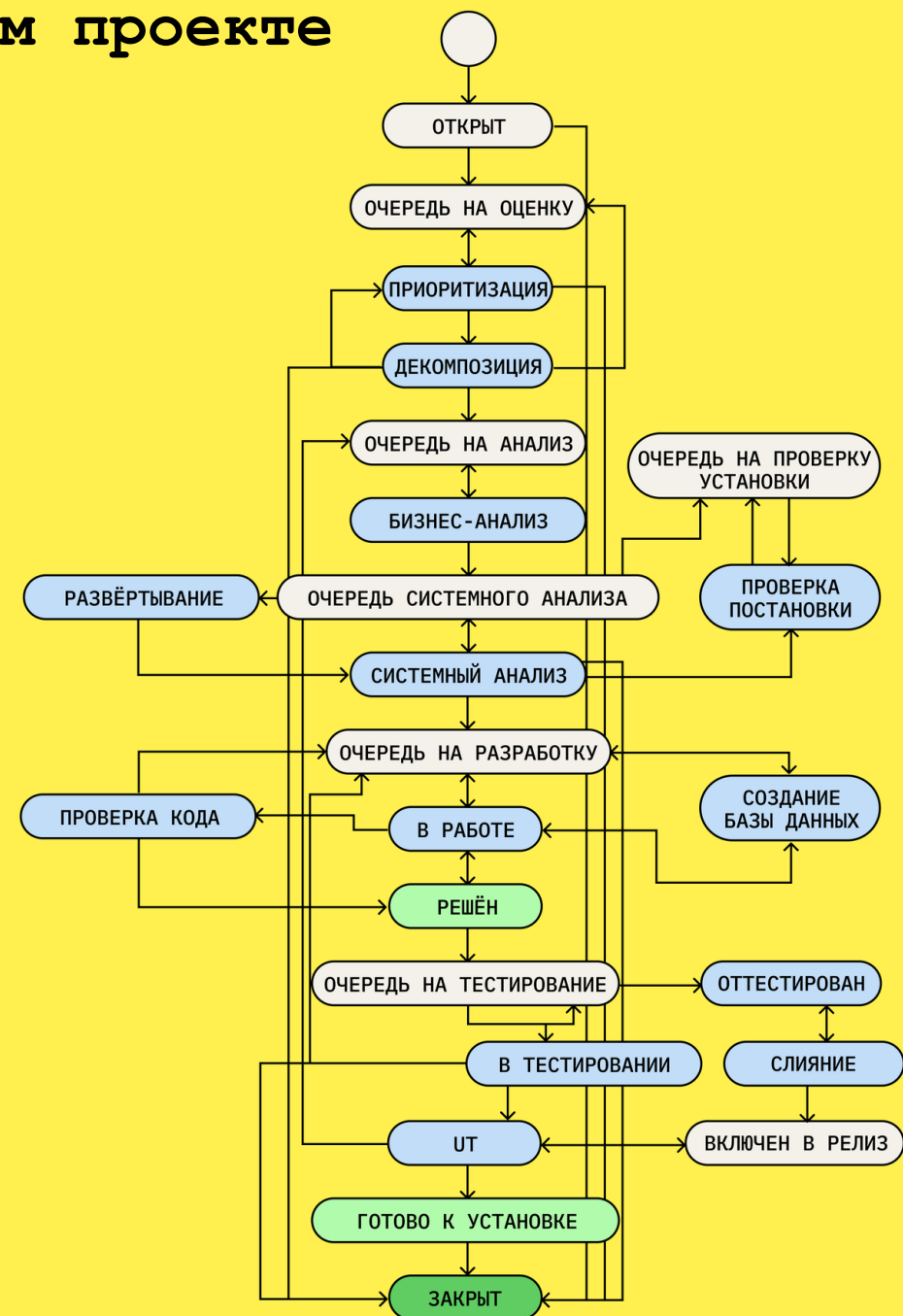
Жизненный цикл бага



Пример жизненного цикла бага на реальном проекте

Допустим, в приложении магазина обнаружили дефект: долго подгружаются товары в каталог. Это может быть связано с тем, что приложение некорректно интегрируется с базой данных товаров. В итоге в каталоге отображаются товары, которых фактически нет, а тех, что в наличии, пользователь не видит. Вот как может выглядеть путь этого бага в Jira:

- Тестировщик описал, в чём проблема, и присвоил задаче статус — **новый баг**.
 - Задачу в работу берёт аналитик, чтобы уточнить, какие условия закладывали в ТЗ для продукта. Баг получает новый статус — **анализ**. На проекте может не быть аналитика или задачу не нужно уточнять — тогда её сразу берёт в работу разработчик.
 - Аналитик добавил уточнения по задаче и передал разработчику. Новый статус — **в разработке**.
 - Тестировщик проводит ретест — проверяет, исправили баг или нет. Если проблему решили, он закрывает задачу, если нет — возвращает задачу разработчику. Она снова получит статус «В разработке».
 - Отработанную задачу тестировщик передаёт во внедрение. После этого приложение либо обновят сразу, либо подождут до релиза: тогда обновлённую функциональность добавят в приложение вместе с другими отработанными задачами.
 - Готово. Теперь пользователь видит только актуальные товары.
- Если дефект повторится, то баг реинкарнирует: его заводят как новую задачу, и он проходит тот же жизненный цикл.



Список статусов бага

Новый. Первый статус в цикле, New, или Обнаружен, означает, что дефект обнаружен тестировщиком, зарегистрирован, и по нему создан баг-репорт, на основе которого разработчик потом будет искать и устранять дефект.

Назначен. Когда новый дефект подтвержден и принят в обработку, получает статус Назначен (Assigned). Как правило назначается ответственный за устранение этого бага (поэтому статус еще может называться «Назначен НА кого-то»).

Открыт. Open-статус означает, что дефект приняли разработчики и начали процесс устранения. На этом этапе возможен переход в статус «Отклонен» или «Отложен», то есть разработчик может «не принять» этот дефект — отклонить или отложить.

Устранен. Или Решен/Исправлен (Fixed, Resolved). Разработчики поработали с кодом, внесли нужные правки, поместили статусом «Исправлен», и возвращают тестировщикам для повторной проверки.

Ожидает повторного тестирования. В статусе Pending Retest дефект ожидает, когда тестировщики повторно проверят его, убедившись что все ОК, код теперь исправлен.

Повторно тестируется. Retest: тестировщик еще раз проверяет этот дефект, и убедившись что он устранен разработчиками, верифицирует это и закрывает дефект, а в противном случае переоткрывает.

Повторно открыт. Если повторное тестирование не смогло устранить баг, обнаруживается снова, ему присваивается статус Reopen. Баг открывается опять и еще раз проходит по циклу.

Проверен. Тестировщик еще раз проверяет (верифицирует) баг, повторно исправленный разработчиком, и если теперь он не проявляется, присваивается статус Verified.

Закрит. Разработчики и тестировщики общими усилиями нашли и устранили дефект, он больше не появляется, и можно присвоить статус Closed.

Отклонен. Разработчик отклонил этот дефект, присвоив статус Rejected, потому что дефект или является дубликатом (уже внесен в систему кем-то из коллег), дефект не воспроизводится, или вообще не считается дефектом.

Отложен. Некоторые дефекты могут счесть не очень важными, не приоритетными, следовательно их можно отложить на потом и устранить в следующих релизах, присвоив статус Deferred и исключив из цикла сейчас.

Дубликат. Этот дефект уже зарегистрирован другим тестировщиком, или суть дефекта та же, тогда присваивается статус Duplicate.

Не дефект. Баг может и есть, но ни на что не влияет, ни в чем не ухудшает функциональность и юзабельность — отмечается статусом Not a Defect.

Не воспроизводится. По какой-то причине баг не удалось воспроизвести, будь то проблемы с платформой, окружением, тестовыми данными, порядком действий и т.п. Присвоен статус Non Reproducible.

Невозможно устранить. Бывают ситуации, когда баг устранить невозможно по какой-то причине: недостатки технологии, стоимость, нехватка времени, недостаточная квалификация или просто лень. Он переводится в статус Can't be fixed.

ОТЧЕТ ОБ ОШИБКЕ (Bug report)

Документ, описывающий и приоритизирующий обнаруженный дефект, а также содействующий его устранению.

ЦЕЛЬ ОТЧЕТА ОБ ОШИБКЕ

- ✓ **Предоставить информацию о проблеме:**
уведомить проектную команду и иных заинтересованных лиц о наличии проблемы, описать суть проблемы;
- ✓ **Приоритизировать проблему:**
определить степень опасности и желаемые сроки её устранения;
- ✓ **Содействовать устранению проблемы:**
предоставить необходимые подробности для понимания сути случившегося, а также анализ причин возникновения проблемы и рекомендации по исправлению ситуации.

РЕЗОЛЮЦИИ ОТЧЕТА ОБ ОШИБКЕ

- | | |
|---|---|
| <ul style="list-style-type: none">✓ Новый;✓ Назначен;✓ Открыт;✓ Исправлен;✓ Повторное тестирование;✓ Проверен;✓ Закрыт;✓ Работает, как задумано (As Designed);✓ Дубликат (Duplicate); | <ul style="list-style-type: none">✓ Невозможно воспроизвести (Can't reproduce);✓ Не будет исправлен (Won't fix);✓ Невозможно исправить (Can't fix);✓ Переоткрыт;✓ Рекомендован к отклонению;✓ Отклонен (Deferred);✓ Отсрочен. |
|---|---|

АТТРИБУТЫ ОТЧЕТА ОБ ОШИБКЕ

МИНИМАЛЬНЫЕ

- ✓ Идентификатор (ID);
- ✓ Краткое описание (Summary, Title)
- ✓ Подробное описание (Description):
 - Шаги для воспроизведения (Steps to reproduce);
 - Актуальный результат (Actual result);
 - Ожидаемый результат (Expected result);
- ✓ Воспроизводимость (Repeatability);
- ✓ Важность (Severity);
- ✓ Срочность (Priority);

АТТРИБУТЫ ОТЧЕТА ОБ ОШИБКЕ

- ✓ Возможность обойти (Workaround);
- ✓ Комментарий (Comment, Note);
- ✓ Среда для воспроизведения бага (Environment);
- ✓ Компонент (Component);
- ✓ Тег (Tag, Label);
- ✓ Ответственный (Assignee);
- ✓ Приложение к багу (Attachment);
- ✓ Срок исправления бага (Due Date);
- ✓ Версия билда, где был обнаружен баг (Affects Version);
- ✓ Версия, в которой должен быть исправлен баг (Fix Version);
- ✓ Создатель отчета об ошибке (Reporter);
- ✓ Оценка трудозатрат на исправление бага (Original Estimate).

Название (заголовок) баг-репорта

- Название не должно быть слишком длинным
- Прочитав название должно быть сразу понятно в чем дело
- Принцип “Что? Где? Когда? WWW (what where when)

<https://www.youtube.com/watch?v=FRSiAl-ydB0>

Плохой пример:

Если открыть вкладку `сrm`, потом выбрать напоминания, потом мышкой нажать на чек-бокс любого напоминания, то тогда не появится корзина.

Хороший пример:

На вкладке `сrm` в разделе напоминаний не появляется иконка удаления при проставлении чек-бокса у любого напоминания.

Что?

Где?

Когда?

Пример:

В заголовке письма не отображаются символы при отправке письма.

В заголовке письма не отображаются символы при отправке письма.

Что?

Где?

Когда?

✓ Так лучше:

В заголовке письма не отображаются русские символы при отправке письма статуса Daily Stat.

Четкое описание позволит избежать неприятных ситуаций

- Лишняя коммуникация
- Время
- Недовольство сторон

Шаги

Описываем все шаги с упоминанием всех вводимых данных (пароли, итоги промежуточных результатов и т.д.)

- Оптимально не более 7 шагов
- Минимально возможные шаги, выкидываем лишнее
- Добавляем пример, на котором воспроизводится баг, если это необходимо (ссылка, файл, фотография и т.д., именно те, с которыми вы поймали баг)

Плохой пример

Шаги

1. Открыть браузер
2. Открыть jivo.ru
3. Войти в систему
4. Вести данные нашего админа
5. Теперь щелкнуть напоминания
6. В напоминаниях создать новую напоминалку
7. Нужно заполнить все нужные поля
8. Сохраняем

Хороший пример

1. Войти в jivo.ru : (логин: oleg@test.ru, пароль: 123456)
2. Перейти в Управление --> CRM --> Напоминания
4. Нажать на кнопку "Создать напоминание"
5. Заполнить поле "Описание" и "Дата" (Например: Test, 04.04.2023)
6. Нажать на кнопку "Сохранить"

Результат/Фактический результат

- Указываем кратко, что произошло и в каком состоянии находится система
- Прикладываем скриншоты, видео, логи (при грамотно составленном баге разработчику достаточно хорошего названия баг-репорта и скриншота/видео/логов)

1. Войти в jivo.ru : (логин: oleg@test.ru, пароль: 123456)
2. Перейти в Управление —> CRM —> Напоминания
4. Нажать на кнопку “Создать напоминание”
5. Заполнить поле “Описание” и “Дата” (Например: Test, 04.04.2023)
6. Нажать на кнопку “Сохранить”

Плохой пример

Результат: Кажется, напоминание не сохранилось и не создано, но вообще должно было

Результат: Сохранение не работает корректно

Хороший пример:

Появилось уведомление об успешном создании напоминания, но напоминание не появилось в списке.

Ожидаемый результат

- В ожидаемом результате указываем по факту, что должно произойти
- Прикладываем скриншот ожидаемого результата.
- Помимо этого прикладываем доказательство, что результат должен быть такой, а не какой то другой



Что служит доказательством:

- ✓ Спецификация
- ✓ Макеты
- ✓ Ссылка на документацию
- ✓ Здравый смысл

1. Войти в jivo.ru : (логин: oleg@test.ru, пароль: 123456)
2. Перейти в Управление --> CRM --> Напоминания
4. Нажать на кнопку "Создать напоминание"
5. Заполнить поле "Описание" и "Дата" (Например: Test, 04.04.2023)
6. Нажать на кнопку "Сохранить"

Фактический результат:

Появилось уведомление об успешном создании напоминания, но напоминание не появилось в списке.

Ожидаемый результат

✓ Плохой пример:

пример

Напоминание должно быть создано

✓ Хороший пример:

Появилось уведомление об успешном создании напоминания,
созданное напоминание появилось в списке.

Итоги

- Название
 - Шаги
 - Фактический результат
 - Ожидаемый результат
 - Дополнительная информация (скриншоты, видео, логи)
 - Приоритет/Серьезность
-

СВОЙСТВА КАЧЕСТВЕННЫХ БАГ-РЕПОРТОВ

- ✓ Тщательное заполнение всех полей точной и корректной информацией;
- ✓ Отсутствие «размытых» формулировок (*«иногда неправильно работает», «UI сломан», «кнопка Export не работает»*);
- ✓ Вся информация, особенно Ожидаемый результат, корректна;
- ✓ Баг заведен на функциональность, которая разработана (*неразработанный функционал за ошибку не считается*);
- ✓ Отсутствует жаргонная лексика (*«Фигово подцепились картинки»*);
- ✓ Отсутствие критики кого-то из участников команды (*«Ну каким болваном надо быть, чтобы такое сделать?!»*);

СВОЙСТВА КАЧЕСТВЕННЫХ БАГ–РЕПОРТОВ продолжение

- ✓ Важность и приоритет бага корректно выставлены;
- ✓ К отчету прикреплены все возможные документы, которые облегчат понимание причины дефекта;
- ✓ Отсутствуют грамматические и семантические ошибки («Not keyboard in parameters accepting values»);
- ✓ Использован правильный технический язык;
- ✓ Краткое описание бага должно следовать правилу WWW (what where when);
- ✓ **В отчете должен быть описан только 1 дефект;**
- ✓ Оформление соответствует соглашению о названиях, принятом на проекте.

Типичные проблемы при описании бага

- ✓ Ошибочные описания и формулировки;
- ✓ Идентичные краткое и подробное описание;
- ✓ Отсутствие фактического и ожидаемого результатов;
- ✓ Лишние пункты при воспроизведении;
- ✓ **Копии экрана целиком, без отмеченной проблемы;**
- ✓ Выдуманные ошибки;
- ✓ Заниженные (завышенные) срочность и важность;
- ✓ Концентрация на мелочах;
- ✓ Указание в шагах воспроизведения неважной информации;
- ✓ Отсутствие в шагах воспроизведения важной информации;
- ✓ Игнорирование последовательных багов.

Плохо сформулировано

Когда мы хотим убрать папку, где что-то внутри есть, оно не спрашивает, хотим ли мы.

- Актуальный результат (Actual result)
- Ожидаемый результат (Expected result)

Хорошо сформулировано

Не производится запрос подтверждения при удалении непустого подкаталога в каталоге SOURCE_DIR.

Act: производится удаление непустого подкаталога (со всем его содержимым) в каталоге SOURCE_DIR без запроса подтверждения.

Exp: в случае, если в каталоге SOURCE_DIR приложение обнаруживает непустой подкаталог, оно прекращает работу с выводом сообщения «Non-empty subfolder [имя подкаталога] in SOURCE_DIR folder detected. Remove it manually or restart application with --force_file_operations key to remove automatically.»

Req: UR.56.BF.4.c.

Плохо

Отправить на конвертацию файл допустимого формата и размера, в котором русский текст представлен в разных кодировках.

Act: *конвертация кодировок производится неверно.*

Хорошо

Отправить на конвертацию файл в формате HTML размером от 100 КБ до 1 МБ, в котором русский текст представлен в кодировках UTF8 (10 строк по 100 символов) и WIN-1251 (20 строк по 100 символов).

Act: текст, который был представлен в UTF8, повреждён (представлен нечитаемым набором символов).

Рекомендации

1. Перечитать баг-репорт сразу после его написания

2. Один баг - одна задача

НО! Баги, относящиеся к одной группе, оформляют в одном баг-репорте

3. Не дублируем баги

4. Исключить переход на личности

5. Локализуем баг

Пример

В форме Авторизации найдено 4 бага:

1. не проходит авторизация
2. едет верстка в поле логин
3. едет верстка в поле пароль
4. едет верстка в кнопке

Оформляем 2 баг-репорта

- баг-репорт на функцию авторизации
- баг-репорт на верстку в полях и кнопке

Локализация

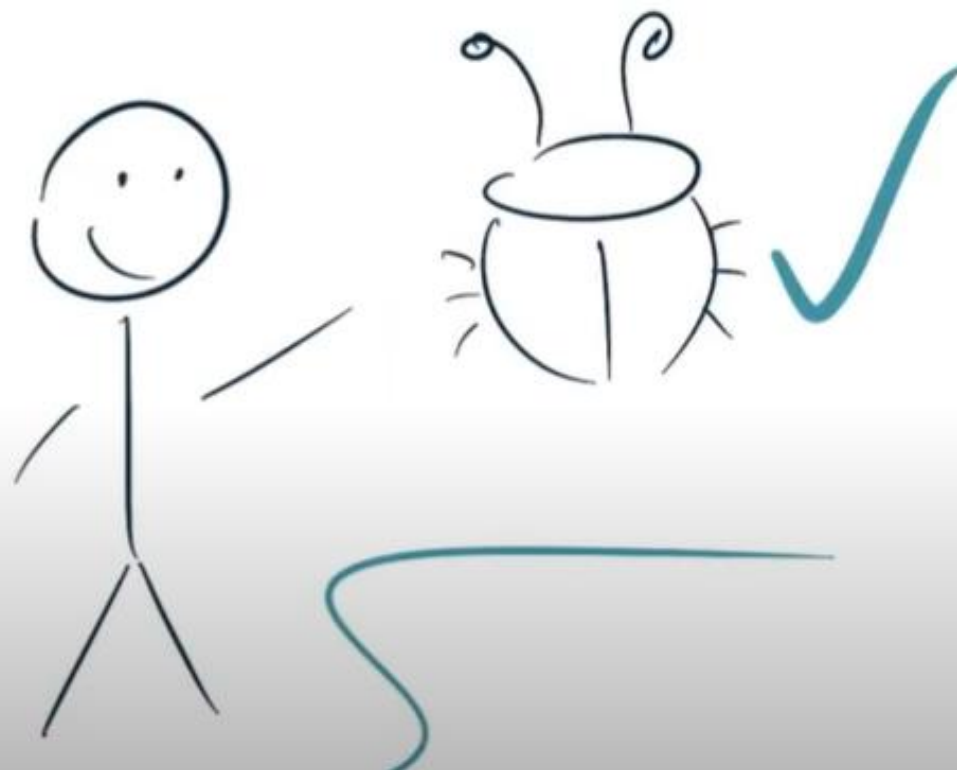
Тестировщик сталкивается с чем-то непонятным в работе системы и начинает разбираться, что произошло. Это называется **локализацией**. Её цель — убедиться, что обнаружили именно дефект. Для этого тестировщик смотрит проектную документацию, ставит эксперименты и узнаёт, в каких ситуациях воспроизводится дефект и можно ли его как-то обойти.

В результате локализации может быть два вывода:

- ✓ Это не баг, или проблема не на стороне разработчиков. Например, внутренний пользователь чего-то не знает по системе и его нужно обучить. Или у пользователя приложения застряли деньги, а проблема на стороне банка.
- ✓ Это баг программы, и его нужно завести в баг-трекинг-системе (оформить баг-репорт).



Не баг, а фича!

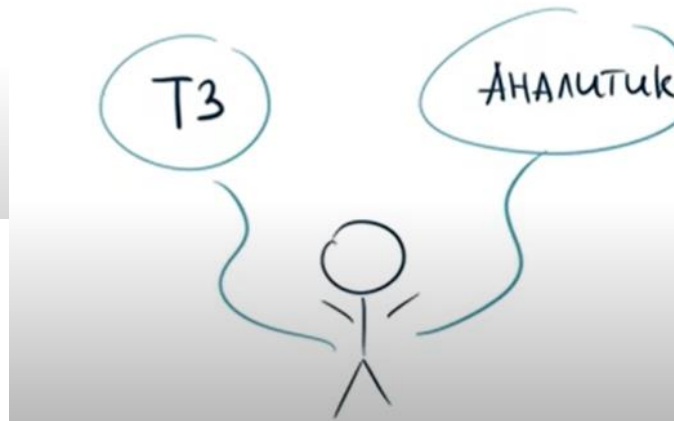


Краткое содержание видео «Не баг, а фича!»

Причины

Ошибка тестировщика
Ошибка в ТЗ
Разработчик ленится
Разработчик сделал лучше

Смотрим в ТЗ
Спрашиваем у аналитика



Варианты решения

- 1) Разработчик исправляет в коде
- или
- 2) Аналитик исправляет в ТЗ

ИНСТРУМЕНТ ДЛЯ ОТЧЕТОВ О БАГАХ

Система для создания, хранения и работы с отчетами об ошибках.

Набор функций:

- ✓ Создание отчётов о дефектах, управление их жизненным циклом с учётом контроля версий, прав доступа и разрешённых переходов из состояния в состояние;
- ✓ Сбор, анализ и предоставление статистики;
- ✓ Рассылка уведомлений, напоминаний и иных артефактов соответствующим сотрудникам;
- ✓ Организация взаимосвязей между отчётами о дефектах, тест-кейсами, требованиями и анализ таких связей с возможностью формирования рекомендаций;
- ✓ Подготовка информации для включения в отчёт о результатах тестирования;
- ✓ Интеграция с системами управления проектами.

MANTIS

MantisHub

mantisbt

vboctor

vboctor

administrator

Issue #

Filters

Advanced Filters

Create Permalink

Viewing Issues

1 - 50 / 2042

Print Reports

CSV Export

Excel Export

Graph

First

Prev

1

2

3

4

5

6

7

8

9

10

11

...

Next

Last

		P	ID	Category	Severity	Status	Updated	Summary
☐			0017964	documentation	text	new	2014-12-13	Documentation - minor typographical error.
☐			0017963	security	major	new	2014-12-13	Error en la Autenticacion
☐			0017918	ui	1 tweak	assigned (syncguru)	2014-12-13	"More Events" Hyperlink in Timeline is not working
☐			0017828	code cleanup	minor	new	2014-12-13	Replace ' ' usages with appropriate constants
☐			0017777	api soap	minor	new	2014-12-13	Add test case for mc_issue_add() and mc_issue_note_add() reporter override
☐			0017460	email	major	new	2014-12-13	Implement a global and generic way of sending e-mail notifications
☐			0016953	email	2 minor	confirmed	2014-12-13	Incorrect usage of \$s_word_separator
☐			0003030	db mysql	4 feature	confirmed	2014-12-13	back slash in search string not escaped
☐			0017910	filters	1 minor	confirmed	2014-12-13	MatchType / Hightlight changed expands all filter fields.
☐			0017852	filters	minor	confirmed	2014-12-13	Tags is showing on its own row in filter box

- Написан на PHP;
- Поддерживает Linux, Windows, Mac;
- Есть time-tracking;
- Разграничение по ролям;
- Отчетность;
- Интеграция с TestLink, Git.

REDMINE

HomeMy pageProjectsAdministrationHelp

Logged as admin - My accountSign out

My project

Search: Jump to a project... ▼

OverviewActivityRoadmapIssuesNewsDocumentsWikiFilesRepositorySettings

Issues

Filters

☒ Status

open ▼

Add filter: ▼

☒ Apply

☐ Clear

#	Tracker	Status	Priority	Subject	Assigned to	Updated
☐ 127	Bug	New	Normal	Ticket with attachments		12/22/2007 12:11 PM
☐ 116	Bug	New	Low	Keep playing audio when rw/ff and preserve pitch.	John Smith	12/17/2007 09:56 PM
☐ 88	Feature	Assigned	Low	HTTP Challenge-MD5 authentication		12/22/2007 04:33 PM
☐ 83	Feature	Assigned	Low	Export the parameters of an input	John Smith	12/17/2007 09:56 PM
☐ 82	Feature	New	Low	Formatted text rendering support	Dave Loper	12/17/2007 06:58 PM
☐ 81	Feature	New	Normal	DVTS support		12/17/2007 06:58 PM
☒ 79	Feature	New	Low	QuickTime capturing		12/17/2007 06:58 PM
☐ 78	Feature	New	Low	Full H323 videoconferencing		12/17/2007 06:58 PM
☐ 77	Feature	Assigned	Low	Closed captions / Teletext support		12/17/2007 06:58 PM
☐ 74	Feature	New	Low	Progressive download playing		12/17/2007 06:58 PM
☐ 73	Feature	New	Low	Dshow tuning support		12/17/2007 06:58 PM
☐ 72	Feature	New	Low	V4L tuning support		12/17/2007 06:58 PM
☐ 70	Feature	New	Low	Electric Program Guide		12/17/2007 06:58 PM
☐ 69	Bug	New	Low	SDL vout cleaning		12/17/2007 06:58 PM
☐ 65	Feature	New	Low	Protocol rollover support		12/17/2007 06:58 PM
☐ 64	Feature	New	Normal	Improve ZLM functionality		12/22/2007 04:33 PM
☐ 63	Feature	New	Low	Gstreamer and Helix integration		12/17/2007 06:58 PM
☐ 62	Feature	New	Low	Gnutella servlet		12/17/2007 06:58 PM
☐ 59	Feature	New	Low	Finalization of Pocket PC port		12/17/2007 06:58 PM
☐ 58	Bug	Assigned	Low	Re-write of the AppleScript bindings		12/22/2007 04:33 PM
☐ 57	Feature	New	Low	MacOS X SVCD support	Dave Loper	12/17/2007 06:58 PM
☐ 51	Bug	New	Low	Better Mozilla plugin control		12/17/2007 06:58 PM

Edit

Status

Priority

Assigned to

Copy

Move

Delete

Immediate

Urgent

High

Normal

Low

New issue

Tracker: ▼

Issues

View all issues

Summary

Change log

Custom queries

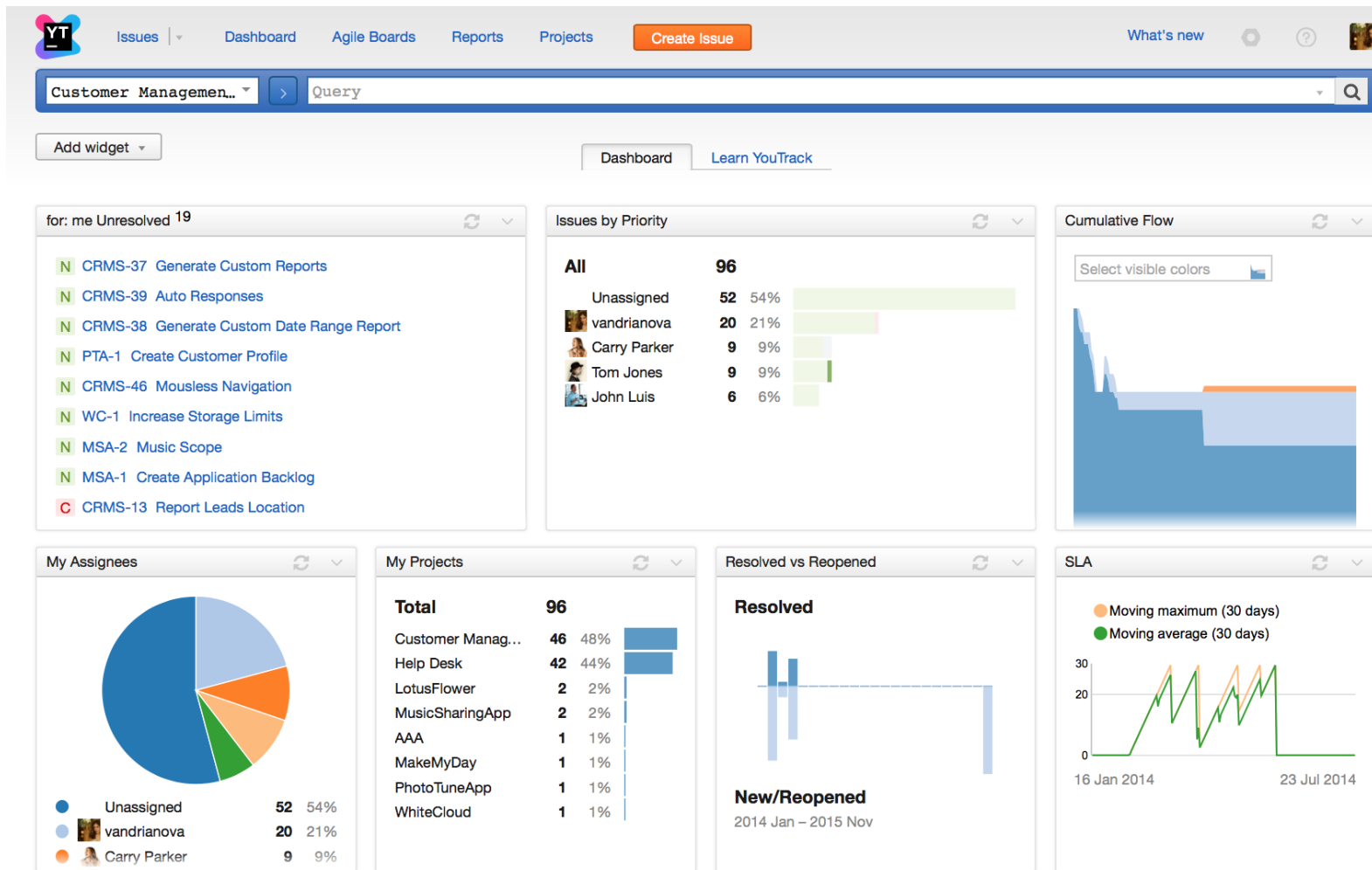
Assigned to me

Due this week

Late features

- Написан на Ruby on Rails;
- Поддерживает Linux, Unix, Mac, Windows;
- Есть time-tracking;
- Разграничение по ролям;
- Интеграция с Git.

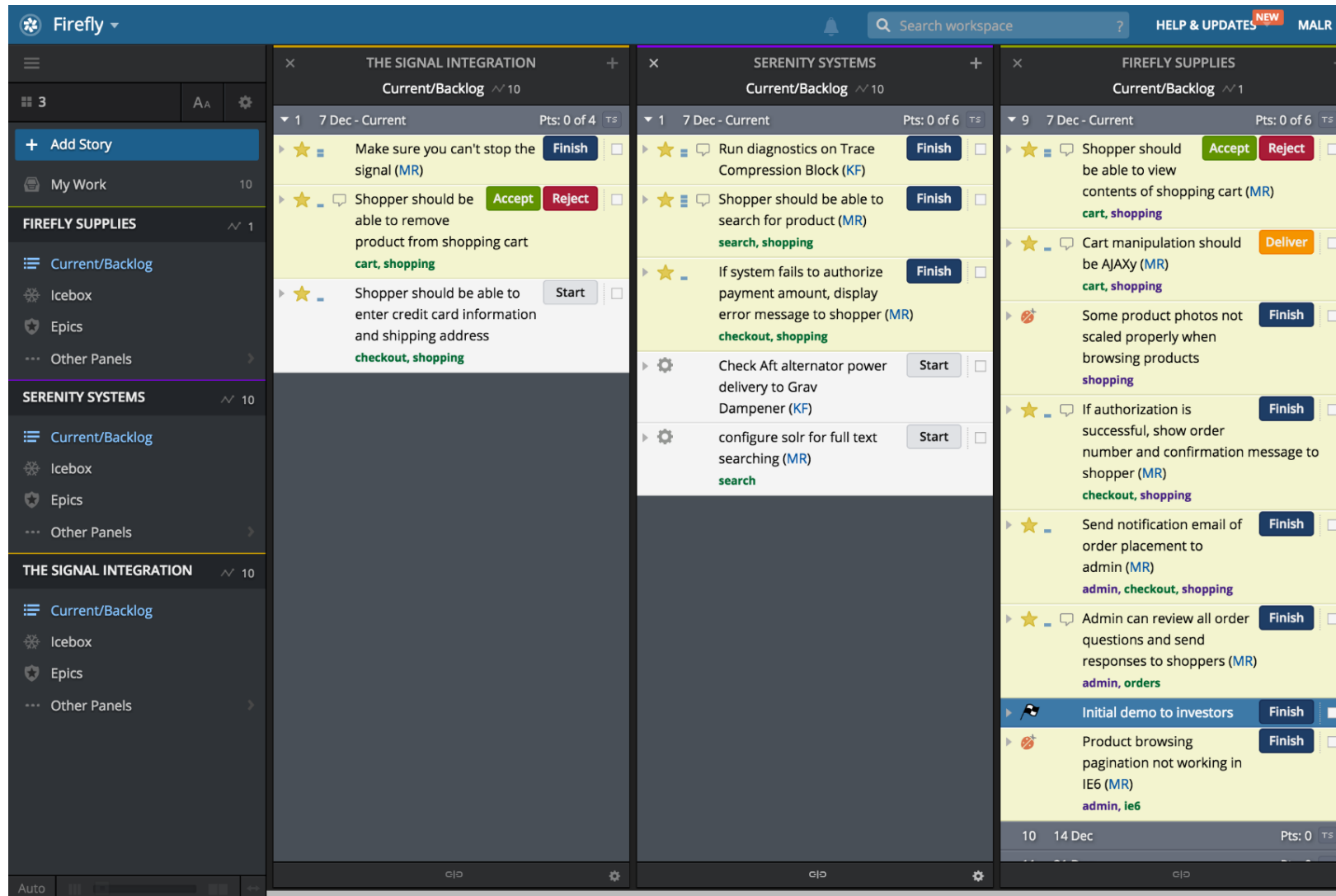
YOUTRACK



- Размещается в облаке;
- Настраивается под Scrum, Kanban;
- Совмещен с управлением проектом;
- Разграничение по ролям;
- Отчетность;
- Есть time-tracking;
- Есть «умный» поиск;
- Есть уведомления пользователей;
- Есть интеграция с тест-кейс системами.

Бесплатный до 10 пользователей

PIVOTAL TRACKER



- Размещается в облаке;
- Настраивается под Scrum, Kanban;
- Совмещен с управлением проектом;
- Отчетность;
- Есть time-tracking;
- Есть «умный» поиск;
- Есть уведомления пользователей;
- Есть интеграция с тест-кейс системами.

Бесплатный до 3 пользователей

ОТЧЕТ О РЕЗУЛЬТАТАХ ТЕСТИРОВАНИЯ

Официальный документ, описывающий результаты, полученные в ходе проведения определенных видов/уровней тестирования.

ЦЕЛЬ НАПИСАНИЯ ОТЧЕТОВ

- ✓ Предоставить статистику по количеству проверок и результатам их выполнения;
- ✓ Предоставить статистику по количеству и серьезности найденных дефектов в приложении;
- ✓ Повысить осведомленность команды о качестве текущей версии приложения;
- ✓ Обратить внимание команды и владельцев бизнеса на области, которые нуждаются в исправлении дефектов
- ✓ Дать рекомендации относительно выпуска или наоборот задержки выпуска версии в релиз.

СТРУКТУРА ОТЧЕТА

1. Уровень/вид проведенного тестирования;
2. Количество пройденных проверок (test cases/test scenarios/checkpoints):
 - Общее количество;
 - Количество успешно пройденных (passed);
 - Количество неудачных (failed);
 - Количество заблокированных проверок (blocked);
3. Количество найденных багов:
 - Общее количество
 - Количество critical багов
 - Количество major багов
 - Количество minor багов
 - Количество trivial багов
4. Области / компоненты, которые наиболее повреждены багами (больше 75% проверок failed)
5. Проблемы, с которыми столкнулись в процессе выполнения задания.

Печальные баги

В 2019 году компания Boeing столкнулась с серьезными проблемами из-за ошибки в программном обеспечении, которое управляло системой MCAS (Maneuvering Characteristics Augmentation System), ответственной за стабилизацию самолета в полете. Эта ошибка привела к крушению двух Boeing 737 MAX и **потере жизней 346 человек.**

В 2009 году **Toyota** объявила о масштабном **отзыве более 4 миллионов автомобилей**, в том числе популярных моделей Camry и Corolla, из-за проблем с программным обеспечением электронных систем управления двигателем. **Баг заключался в том, что система контроля двигателя иногда неправильно интерпретировала сигналы от датчика педали газа, что могло приводить к неожиданным и опасным тормозам.** В результате этой проблемы произошло несколько аварий, в том числе несколько летальных случаев. Этот инцидент стал одним из **самых крупных отзывов автомобилей в истории.**

в 2015 году было обнаружено, что в некоторых автомобилях Jeep Cherokee с системой управления двигателем была уязвимость, позволяющая злоумышленникам удаленно получить доступ к системе управления и даже контролировать движение автомобиля. Это привело к масштабному отзыву более 1,4 миллионов автомобилей Jeep Cherokee

У БАГОВ ТОЖЕ ЕСТЬ ЧУВСТВА

НАЙДЯ БАГ:
СООБЩИТЕ О НЕМ

БАГИ НЕ ЛЮБЯТ
БЫТЬ ЗАБЫТЫМИ



НАЙДЯ БАГ:
ИЗУЧИТЕ ЕГО

БАГИ ЛЮБЯТ
БЫТЬ ПОНЯТЫМИ



НАЙДЯ БАГ:
СДЕЛАЙТЕ СНИМОК

БАГИ ЛЮБЯТ ХРАНИТЬ
ПАМЯТЬ О ВСТРЕЧЕ



НАЙДЯ БАГ:
УЗНАЙТЕ ЕГО ДРУЗЕЙ

БАГИ ОБЩИТЕЛЬНЫ



НАЙДЯ БАГ:
СООБЩИТЕ О НЕМ БЫСТРО

ИНАЧЕ БАГИ ОБЖИВУТСЯ И
ПОСТРОЯТ СЕБЕ ДОМА



НАЙДЯ БАГ:
БУДЬТЕ ЧЕСТНЫ

БАГИ НЕ ЛЮБЯТ
СПЛЕТЕН



НАЙДЯ БАГ:
ЗАПОМНИТЕ КАК ВЫ
ВСТРЕТИЛИ ЕГО

БАГИ РОМАНТИЧНЫ



НАЙДЯ БАГ:
НЕ ИГНОРИРУЙТЕ ЕГО

БАГИ МОГУТ УКУСИТЬ,
ЕСЛИ ИХ НЕ ЦЕНИТЬ

